EXTENDING GRAPHICAL COMPOSITION FROM IMPROVISATION IN MUSIC

Author: William Morris

Supervisor: Simon O'Keefe
Date: 10/09/2017

Project report submitted for the degree of MSc in Information Technology from the Department of Computer Science, University of York

Word Count: 13,331 as counted by Microsoft Word. This includes the title page, abstract, table of contents and body of the report but not the references or the appendix.

ABSTRACT

This project explores whether the construction of visual-interactive systems can be accelerated for end-user development. It focuses on the human processing capabilities derived from conventional attributes of improvisatory performance. This is accomplished by the design of a real-time system that uses MIDI as a source of tracking improvisatory techniques in music to extend graphical composition capabilities for *Max*, a hybrid development environment for visual programming. Congruous fields of research and development in the literature are researched and reviewed to inform the development of a series of software modules for *Max*. These modules are engineered to carry out specific operations pertinent to improvisatory performance and graphical composition. The findings reveal a more systemic approach to visual-interactive systems through design abstractions, and expose a series of future considerations related to the separation of improvisatory and graphical functionality.

TABLE OF CONTENTS

1 Introduction 5
1.1 Research Context 5
1.2 Keywords 5
1.3 Overview of the Report 5
1.4 Project Methodology 6
1.5 Project Timeline 7
1.6 Summary 7

2 Literature Review 8
2.1 Introduction 8
2.2 Hybrid Environments for Real-Time Visual-Interactive Systems 8
2.3 The *Max* Interactive Development Environment 9
2.4 Abstraction in visual-Interactive Systems 9
2.5 Abstraction for User-Interface Design 10
2.6 Alternative Visual-Interactive Technologies 10
2.7 Analysis in Visual-Interactive Systems 11
2.8 Extending Analysis for Visual-Interactive Systems 11
2.9 Conclusion 12

3 Requirements 13
3.1 Introduction 13
3.2 Requirements Process & Elicitation 13
3.3 Deriving Improvisational Requirements 13
3.4 MIDI Standards 15
3.5 Computer-Graphics Standards 16
3.6 Requirements Specification 18
3.7 Summary 20

4 Design 21
4.1 Introduction 21
4.2 Design Principles 21
4.3 the Design of the System 23
4.4 Key issues in the design 25
4.5 User interface design 26
4.6 Summary 26

5 Implementation 27
5.1 Introduction 27
5.2 Constructing with Max 27
5.3 Module descriptions 28
5.4 The [kl.mfilter] Module 29
5.5 The [kl.nodec] Module 31
5.6 The [kl.FpWorld] Module 34
5.7 Summary 35

6 Evaluation 36
6.1 Introduction 36
6.2 Requirements Validation and Verification 36
6.3 Design Evaluation and integration 38
6.4 Summary 38

7 Conclusion **39**
7.1 Introduction 39
7.2 Impact 39
7.3 Future Considerations 39
7.4 Reflection 40
8 Bibliography **41**

1 INTRODUCTION

1.1 RESEARCH CONTEXT

Interactive systems are defined by the interaction between the human and the computer. This broad definition encapsulates a wide-reaching assortment of systems, including games, web-browsers and interactive development environments (IDEs). For all interactive systems, it is essential to understand human processing capabilities for information processing. This project focuses on visual-interactive systems - systems that emphasize the use of visualization or visual analytics. More specifically, this project explores visual-interactive systems in combination the human processing capabilities in music. As such, these types of systems are commonly termed audio-visual systems, multimedia-systems or video-synthesizers interchangeably.

Research to date surrounding visual-interactive systems has led to the creation of more specialized development technologies. These languages and frameworks accommodate building these systems, and more performance-intensive interactive systems have often led to the emergence of new hardware technology. The staggered relationship between the two, where one continually drives the other, is best exemplified with the release of ambitious video games, resulting in a market for newer graphics hardware. The overreaching problem this project is based upon is how software opportunities can be provided that drive hardware to its threshold, and simultaneously provide end-users with more interface interaction to minimize the numerous software development complexities that exist within interactive systems.

1.2 KEYWORDS

This project provides research into the acceleration or facilitation of visual-interactive systems by exploring the disciplines of musical improvisation; interactive and audio-reactive visual systems; data & statistical analysis; computer art; MIDI; Audio-analysis; Max and OpenGL.

1.3 OVERVIEW OF THE REPORT

This report is structured into chapters and sections that are arranged to step through the development process:

Chapter 1: The *Introduction* introduces the reader to the research context, the project methodology and a brief timeline of the project.

Chapter 2: The *Literature Review* examines some of the primary cases surrounding this project. These include hybrid environments for real-time visual-interactive systems, the Max interactive development environment, alternative frameworks and environments, abstraction in visual-interactive systems and data analysis in visual-interactive systems.

Chapter 3: The *Requirements* introduces the engineering cycle phase by reporting on the requirements process and elicitation, the domain knowledge leading to the requirements specification itself.

Chapter 4: The *Design* analyses the most recognised design principles that featured as a result of the derived requirements, providing Max-based UML for outlining the system at architectural and behavioural level. It includes a table with the modules featured in the system. **Chapter 5**: The *Implementation* Chapter introduces a context to Max's syntax and the implementation of three of the modules, inclusive of how each module's features practical considerations, construction and testing in relation to the system's design and requirements phases.

Chapter 6: The *Evaluation* evaluates the system by validation of verification of each requirement, along with an evaluation of the system design.

Chapter 7: The *Conclusion* concludes the project report, and enumerates potential future expansions for the design and overall impact.

1.4 PROJECT METHODOLOGY

An *Iterative design* methodology included the use of continual prototyping, testing and refining the system. The reasons that influenced this methodology include:

The reasons behind of such a methodology are interconnected; they include, but are not limited to:

- Research into relevant technologies to expose system potentials;
- Regular interaction with my supervisor to consult on how the project should evolve;
- Regular confrontations and analysis of system behaviours that lead to fundamental changes in the system's design;
- Change in any decisions throughout the process was often not expensive, especially in the earlier stages of development;
- Changing the engineering direction in regard to scope or testing inconsistencies.

A more specific project lifecycle model for this design methodology can be seen in the diagram below. The number of iterations throughout the project was difficult to quantify, however this report illustrates three iterations or 'phases' in the requirements, design and implementation chapters.

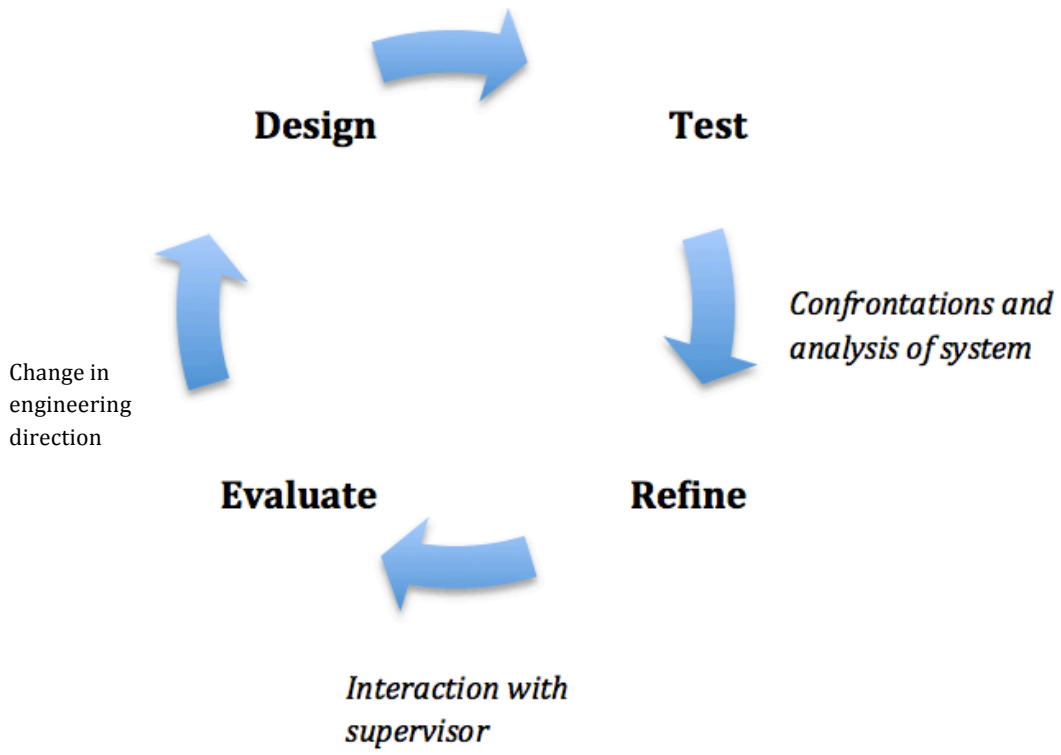


Figure 1: Diagram portraying the methodology of the project

1.5 PROJECT TIMELINE

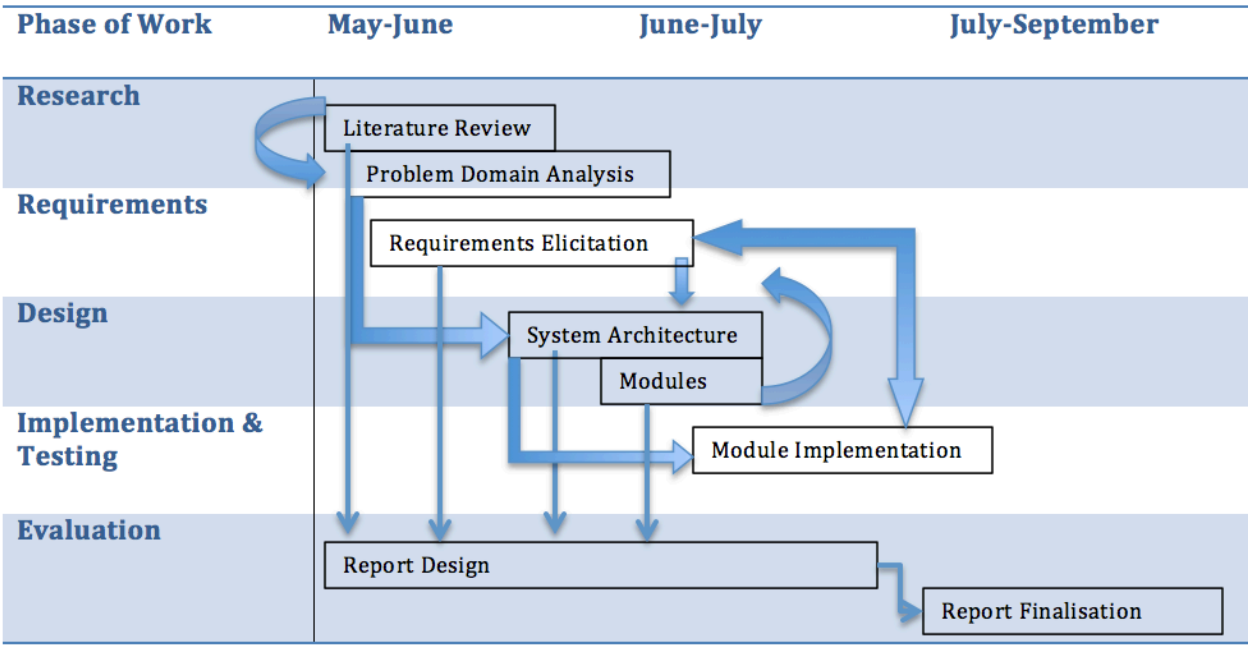


Figure 2: The arrows indicate a one or two-way relationship back and/or forth to illustrate the iterative processes involved between the various stages of the project.

Some but not all of the research carried out for the L*iterature Review* led to the identification of relevant problem domain headings for the project, as examined in the *Requirements* Chapter. The domains influenced the requirements specification that led to the design of the system's architecture mid-project. Continuous module design prototyping sometimes derived new or modified existing requirements too, in combination with regular supervisor consultations. Requirements derivation became less frequent throughout the implementation phase. With a series of drafts and notes and implementation nearly complete at the end of June, emphasis was placed on report finalisation mid-July.

1.6 SUMMARY

The next chapter provides the relevant cases, backed by credible literature to expose the software potentials and gaps that lay the foundations of the project. These include: hybrid environments for real-time visual-interactive systems; the Max interactive development environment; alternative frameworks and environments; abstraction in visual-interactive systems and data analysis in visual-interactive systems.

2 LITERATURE REVIEW

2.1 INTRODUCTION

The Research Context in 2.2 of the *Introductory* Chapter claimed that new software potential for visual-interactive systems existed, exploring the human processing capabilities in regard to improvisation in music. This is by no means the first project that is researching music improvisation as a human processing capability for a visual-interactive system, but this review attempts to expose pertinent new potentials or alterations for these types of systems. The review highlights the importance of hybrid environments for real-time visual interactive systems, notable technologies in visual-interactive systems, and the importance of abstraction in visual-interactive systems and user-interfaces. To demonstrate relevance, the following literature review continually attempts to expose these potentials, as the acting foundation behind the project's motivations and engineering requirements.

Because of the scientific nature of this report and the immediate relevancy in regard to the engineering carried out in this project, it is important to note at this stage that the literature review does not provide research into *music improvisation* as a musical construct. But *Improvisation: Its Nature and Practice in Music* by Derek Bailey [1] serves as valuable reading for the nature of improvisation in all its forms. Briefly however, music improvisation can be defined as allowing the creative activity of immediate musical composition, to which it combines the communication of emotions and instrumental techniques along with an immediate reaction to other musicians. It becomes more abstract when this is further elaborated more on this. The *Requirements* Chapter discusses this further in relation to the engineering potentials and considerations carried out in the project.

2.2 HYBRID ENVIRONMENTS FOR REAL-TIME VISUAL-INTERACTIVE SYSTEMS

Historically, many have interpreted the 'visualisation' aspect in common interactive systems as the subsequent means to view the results from data analysis, but as Stalling et al. observes in the introduction to his report *Amira: a Highly Interactive System for Visual Data Analysis,* 'while in such integrated systems visualization is usually just an add-on, there are also many specialized systems whose primary focus is upon visualization itself.' [2] This is especially the case for those who are involved in *live-coding*, sometimes referred to as on-the-fly programming - a performing arts form centred upon the writing of source code and the use of interactive programming in an improvised way.

It is evident that there is a dedicated need for a live-coder to be able to make modifications to a program without halting its on-going execution. It would not be convenient for an audience if a visualisation had to be re-executed so the performer could add another graphical element or sound to his/her code. The optimal solution to this is a hybrid-interactive development environment for visual-interactive systems. Whilst there are many solutions that have been embraced in the performance of music for live-coding such as SuperCollider[1], there have been fewer contributions to this for the live-coding of visual-interactive systems for computer graphics. With that said, there are academically recognised environments centred on building visual-interactive systems that operate in real-time.

[1] An environment and programming language originally released in 1996 by James McCartney for real-time audio synthesis and algorithmic compositions.

These include openFrameworks[2], Cinder[3] and Max[4]. The most distinguishable in regard to its large community size and development methodology is Max, a visual-programming language.

2.3 THE *MAX* INTERACTIVE DEVELOPMENT ENVIRONMENT

As software that has been around for over three decades, *Max (*formally Max/MSP) informally descends from the language MUSIC-n to house live-coders not familiar to certain design principles like scoping, name-spacing or object-orientated programming. *Max* is a visual-programming language written for music and multimedia, is modular by design, and encompasses routines that exist as shared libraries. It encompasses interaction with MIDI and audio, providing several high-level objects, as well as many OpenGL objects within the Jitter portion of *Max* that provide real-time video and matrix processing for 2D and 3D graphics. Most end-users regard *Max* as a visual programming language, working at high level, but there are low-level operations that schedule the real-time management of communications between objects[5] in the modular fashion of outlet-to-inlet or output-to-input.

On their paper on *Interactive Systems for Visual Data Analysis*, Stalling et al. make some pertinent points on visual programming languages in visual-interactive system – more specifically on the benefits of modular approaches to interactive systems in that 'these systems (interactive systems) are not targeted to a particular application area, but provide many different modules which can be combined in numerous ways, often adhering to the data-flow principle and providing means for visual programming' [2]. They subsequently mention that 'in these ways, custom pipelines can be built to solve specific visualization problems. Although these visualization environments are very flexible and powerful, they are usually more difficult to use than special-purpose software. In addition, a major drawback induced by the data-flow principle or pipelining approach is the lack of sophisticated user interaction.' [2]

Max is modular, and end-users can draw learning parallels with physical modular systems such as *Buchla Series* or the *Korg MS20* to gain an overview of how modular patching works in *Max*. In addition, as long as the components are readily available, modular patching is instant, flexible and highly customizable – to accommodate the necessary features of musical improvisation. However, although Max supports improvisation and real-time hybrid development useful for on-the-fly programming, deploying a visual-interactive system in this environment invites a series of engineering complexities - complexities can be solved through abstraction paradigms. The *implementation* Chapter discusses the relevant *Max* syntax in more detail.

2.4 ABSTRACTION IN VISUAL-INTERACTIVE SYSTEMS

The use of abstraction in visual-interactive systems is so that higher level programming can be adopted. As Gomes and Velho note in their paper *Abstraction paradigms for computer graphics*, 'This abstraction step becomes very important when we are working on a very interdisciplinary area such as computer-graphics that uses methods from geometry, algebraic geometry, analysis, discrete mathematics, and so on.' [3] Furthermore, in his book *Computation Visualization: Graphics, Abstraction and Interactivity*, Strothotte notes that the importance of abstraction within visualization is being able to 'separate out important features from less important ones, and to make this choice evident for the viewer.' [4]. For visual-interactive systems, especially in gaming, many mathematical models are used to simulate real-

[2] An open source toolkit designed for "creative coding". OpenFrameworks is written in C++ and built on top of OpenGL
[3] An open-source programming library designed to give C++ language advanced visualization abilities
[4] A visual programming language
[5] Entities in which structured data is passed through

world objects and 'in order to understand these models and pose relevant problems in each particular field of this area, it is important to create levels of abstraction to encapsulate common properties of the different models and allow us to have a global conceptual view of the methods and techniques in each field.' [3] Since then, there have been continual attempts to solve the issue of abstraction, through the development of new technologies – frameworks, languages and softwares. Nonetheless, 'The creation of adequate abstraction levels allow us to search for better mathematical tools to tackle the problems in an objective way in each level.' [3]

2.5 ABSTRACTION FOR USER-INTERFACE DESIGN

On their article on *Visual Interactive Systems for End-User Development: A Model-Based Design Methodology*, Constabile et al. observe that 'On the whole, there is a communication gap between designers and end-users, originated by their different cultural backgrounds.' They go on to mention that the designers and end-users 'adopt different approaches to abstraction since, for instance, they may have different notions about the detail that can be abridged. Moreover, end-users heuristically reason rather than algorithmically, using example and analogies rather than deductive abstract tools; they document activities … End users retain distinct types of knowledge and follow different approach and reasoning strategies to modelling, performing and documenting the tasks to be carried out in a given application domain.' [5]. Identifying the most fitting users for heuristic evaluation is crucial, as these types of systems are niche and open to a lot of artistic objectivity, but should nonetheless be inclusive of graphic designers, musicians and developers with expertise in visual-interactive systems.

A small but noteworthy software potential to be aware of when designing interactive systems is related to the mapping of parameters to hardware interfaces, commonly referred to in music technology as *parameterization*. This becomes an ever more pertinent problem in regard to these systems when they operate in real-time environments for on-the-fly live programming.

2.6 ALTERNATIVE VISUAL-INTERACTIVE TECHNOLOGIES

In 2000, The Khronos Group focused on the creation of open standard, royalty-free application programming interfaces as graphic-based technologies to provide developers with the means to developing computer-graphics. These include OpenGL, standing as one of the major graphic APIs to date, WebGL supporting web-based technologies and OpenCL, a framework for programs that execute across heterogeneous platforms. However, these APIs were frameworks that required substantial low-level engineering and maintenance.

In response to this, in 2005, Zachary Lieberman and a group of members of the openFrameworks community constructed openFrameworks as an open-source toolkit designed to 'assist the creative process by providing a simple and intuitive framework for experimentation.' It was written in C++, and built on top of OpenGL. This provided higher-level abstraction that concealed some of the low-level function definitions that exist in OpenGL's API, and provided a community in which end-users could begin to develop visual-interactive systems. Although popular and effective for building complex visual-interactive systems, openFramework's API documentation is poorly written with many of its functions not described [6], which can slow down the engineering process. The alternative was to consult the online-community, which is fairly popular, but there was no clear support that I could discover directly related to building visual-interactive systems using music improvisation as a data source.

Another recognised visual-interactive technology is Cinder, a C++ library. This language supports domains for graphics, audio, video, and computational geometry. Cinder is cross-platform, with official support for macOS, Windows, Linux, iOS, and Windows UWP. Cinder is production-proven, powerful enough to be the primary tool for professionals, but suitable for learning and experimentation. With

that said, Cinder API documentation is also vague and needs contributions from the open-source community.

2.7 ANALYSIS IN VISUAL-INTERACTIVE SYSTEMS

For a performance with a visual-interactive system, one of the more effective means to sourcing data for live-coding or on-the-fly programming is through the analysis of music from operations on one or more audio signals.

These operations can be simple mathematical models that perform a procedural or routine set of functions, and more complex to employ the use of fast-Fourier Transform (FFT), which converts a signal from its original domain (often time or space) to a representation in the frequency domain and vice versa. There are a host of subsequent algorithms that can be performed on the frequency domain thereafter, including the Cooley-Turkey FFT or the Prime-factor FFT algorithms. Depending on the outcome written by the end-user, many of these audio-based operations can affect visual representations in computer-graphics, by being scaled and/or converted into more valid data-types for matrix processing.

There is a growing community with notable contributions surrounding visual-interactive systems that employ the use of audio analysis. To name a few that use some of the recognised frameworks and technologies, designer Memo Akten built a large interactive musical laser forest that premiered at the STRP Biennale in Eindhoven – 'this giant interactive forest covered almost 450 square meters and was composed of over 150 musical 'trees' made of rods and lasers. The audience could freely explore the space, physically tapping, shaking, plucking, and vibrating the trees to trigger *audio*.' [7]. London-based company *Field* built an art installation called *Communion,* in which minimal graphical organisms evolve and react to polyrhythmic *audio* [8]. Cycling 74, the authors of Max released the Max package VIZZIE, in which a series of modules use *audio* to affect the 2D representation of image through matrix processing. [9]

Whilst using *audio* analysis for visual-interactive systems has on the whole been well received, its disadvantages should not be overlooked. More ambitious use of data-analysis like Fast-Fourier Transform (FFT) is CPU expensive, and in concurrence with any graphical processing can result in a system that could likely cause lag or crash throughout a performance. This often means that the use of concurrency, parallel processing or aspects of heterogeneous computing needs to be considered when engineering visual representations within the system.

2.8 EXTENDING ANALYSIS FOR VISUAL-INTERACTIVE SYSTEMS

Audio's counter-part, MIDI has remained largely unobserved as a source of analysis for visual-interactive systems. It is relatively simple, perhaps too simple for analysis, and more extensive analysis should be done to employ this data-type as a source. However, MIDI is more lightweight, more user friendly, and has the potential to drive visual-representations in computer-graphics. In addition, whether a visual-interactive system employs analysis that is sourced from audio or MIDI, it is nonetheless concealed to an audience, making a case for MIDI as a more appropriate source for developer – especially a developer with limited knowledge of programming and/or the mathematics related to audio-analysis.

Moreover, the use of such a lightweight source can affect change in CPU cost, creating more idle CPU to implement tasks in machine learning. On the subject of systems for music information retrieval, Fiebrink notes that *'Classification is a lively area of research and encompassing feature weighting attempts to further optimize a classification scheme by assigning real-valued weights to features*

according to their relevancy.' She goes on to note that *'Music classification continues to be an important component of music information retrieval research. - an 'underutilized tool for improving the performance of classifiers is feature weighting.'* [10]

In their paper on Machine Learning of Musical Gestures, Caramiaux and Tanaka present an overview of machine learning techniques with regard to its application in interactive music and new instrument design is presented to the non-specialist reader. More importantly – how machine learning is useful for creating expressive musical interaction and in turn 'why live music presents a pertinent and challenging use case for machine learning.' [11]

Other contributions include an external Max package called [ml-lib] inclusive of common supervised, unsupervised and reinforced learning tasks [12]. In addition, Rebecca Fiebrink has created the standalone application 'Wekinator', which 'provides a set of graphical interfaces for users to iteratively create training examples through real-time demonstration, apply supervised learning algorithms to create trained models from those examples'. [13]

2.9 CONCLUSION

The possible software potentials in visual-interactive systems that use the human processing capabilities for improvisation in music pose a series of pertinent issues regarding an effective engineering process. Furthermore, extending graphical composition using improvisation in music is used as a problem in an attempt to address and evaluate the three cases this literature review above raises. The cases raised in this review were not considered separately, and raised as a result of the iterative nature of this project set out in 2.4 of the *Introduction*.

The first case argues Max as a suitable hybrid environment for real-time visual-interactive systems against alternative frameworks such as Cinder or OpenFrameworks. Through exploration and research into alternative frameworks, the crowning argument against using these is the lack of API documentation, resulting in a slower engineering process that would be detrimental to the timeline of the project.

The review then makes the case for using abstraction as a software design principle in order to hide the complexities in computer-graphics for user-interface and separated functionality, notably for the OpenGL language, which Max is capable of hosting. In order to achieve viable levels of abstraction within a visual-interactive system, you should modularise components of a system by continuous prototyping and observation into software and end-users via heuristic evaluation. Similarly, other gaps for further research include similar topics in human processing surrounding the use abstraction for computer-graphics, especially surrounding 3D vector-based graphics in OpenGL. These include camera tracking, controller mapping and generative computer-art.

The final case for the literature review is the use of MIDI against audio as a viable source for visual-interactive systems. MIDI was explored in the context of analysis for visual-interactive systems, arguing some of the potential it has as an under-contributed area in these systems. MIDI can be a more useful source of data for driving graphical operations in these systems as it is lightweight, but it is important to be aware that the lack of sourced data requires more extensive data analysis from human processing capabilities in music improvisation to be carried out in order to extrapolate more data for an OpenGL context. The following chapter takes these cases into consideration, and defines a series of domain problems surrounding the requirements process and specification for designing an appropriate system.

3 REQUIREMENTS

3.1 INTRODUCTION

This section describes the requirements process and elicitation by outlining the potential stakeholders of the system. From then, improvisational requirements are drawn, and the two primary problem domains are discussed so that requirements can be specified. The illustrated requirements specification in 3.6 is laid out to demonstrate the iterative nature of requirements derivation through each phase.

3.2 REQUIREMENTS PROCESS & ELICITATION

The requirements elicitation was a two-constituent process. The musical constituent of this project involved extensive research into defining improvisatory performance for an understanding of how attributes of improvisatory can be re-enacted by a system or not. The visual constituent of this project involved extensive research into computer art, along with video, graphic and image installation pieces.

As well as deriving requirements stemming from the initial description of the original project proposal itself, some the overall system design requirements were derived from continuous prototyping with notable technologies such as OpenFrameworks and Max, and the observation of other software products features and interface, including *VDMX*[6] and Apple's *QuartzComposer*[7]. Early on throughout the iterative SDLC, stakeholders with both direct and indirect interaction with the software of this project were perceived to take one or more of the roles listed below.

Direct stakeholders include:

- *Musicians* – Primarily, the system was purposefully built to accommodate stage performers who are physically or mentally too preoccupied to deal with audio-visuals;
- *Graphic Designers* – Although intended to accommodate the role of musicians, the system should serve as a means to producing images or graphics for computer-art;
- *Developers* involved in using or modifying system to fit with their own systems.

Indirect stakeholders include:

- *Consumers* - with no access to the system, only a visible access to any video produced by the system, in real-time and not.

For clarity, for the remainder of the report the stakeholder(s) will be referred to collectively as the end-user(s).

3.3 DERIVING IMPROVISATIONAL REQUIREMENTS

The main goal was defined as building an interactive system capable of detecting or reacting to improvisatory performance. In order to derive a set of suitable requirements from this, it was important to define improvisation for a requirements context. This proved troublesome, and even though I am from a musical background, many musicians including myself have difficulty defining 'improvisation'. Instead, it was more effective to describe how to approach it. Within a performance,

[6] Audio-visual software that uses layering, MIDI and OSC to create visual effects for live performance.
[7] A node-based visual programming language provided as part of the XCode development environment.

musician(s) who improvise can be described as employing the following attributes: to *listen, react, augment*[8] and *create*.

The system should be able to interpret these attributes to impact the graphics the system creates. In order to ease the requirements process, I personified the system by acknowledging that the system should do the same as any human who is attempting to improvise; the system should be able to *listen, react, augment* and *create*. This way, requirements of the system could be derived from a more human context. Albeit more abstract, the approach was more user-focused, and took less of a perspective of current technologies and how requirements could be derived to expand their current features. For a system to harness the attributes of improvisation, there is a desire for it to be capable of tracking historical patterns for recognition of improvisation methods. This should be done through the collection and storage of a series of notes for data analysis.

In order to do this, the requirements needed to be constrained by acknowledging the scope and timeframe allocated for this project. There are two reasons for this. The first is that these improvisational attributes listed are nebulous; too many requirements could be derived from them, resulting in a system too conceptual to approach. In addition, improvisation in music can be extremely complex, and has little or no boundaries in how it should be performed. Basically, there are no rules. This makes engineering an interactive system to detect improvisation for all possible scenarios problematic.

RHYTHMIC INTERPRETATION

With that said, a specific element of improvisation was chosen for the system so that its scope could be narrowed – rhythmic interpretations in improvisation. Often argued to be the basis of improvisation, rhythm makes up one of the foundational pillars of improvisatory performance, along with pitch, dynamics and timbre[9]. It was therefore decided early on throughout the project's lifecycle that the system would detect more focused rhythmic aspects of improvisatory performance that adhere to the attributes of improvisation listed above in 3.3.

TECHNOLOGICAL CONTEXT

Presently, instrumentation falls into two dividing categories: Acoustic and electronic instruments. The retronym 'acoustic music' was used to distinguish it from electronic music after the invention of the synthesizer. The invention of MIDI in 1984 has redefined how the direct stakeholders listed in 3.2, notably musicians, use MIDI in a DAW (digital-audio-workstation) to record and perform live. This changed the context in which improvisation was performed. Nowadays, using MIDI to record and perform parallels the traditional approach of recording audio in mono or stereo format. The decision to use MIDI as a format was decided at the beginning of this project, but being aware of both formats was important to gain useful insight into what other competitive forces were present, especially when making observations from other software products.

To use MIDI as a technology for this system, it was important to observe the foundations of any relevant domain knowledge by acknowledging and defining the relevant standards so that the requirements being engineered were well grounded.

[8] The process of adding a sound to any fragment of what others were doing.
[9] That is, the perceived sound quality of a musical note, sound or tone.

3.4 MIDI STANDARDS

MIDI (Musical Instrument Digital Interface) is a communications protocol that receives information messages from input devices. MIDI is widely used by artists who are using DAWs (digital-audio - workstation) like Ableton Live or Logic Pro to be able to connect physical interfaces, samplers, synthesizers, or external/internal software VST[10] instruments too.

Midi messages are in binary format, in 3 eight-bit bytes. Information is stored in 7-bits [0 – 127]. The most significant first bit is for the messages 'status', while the other two represent its 'data'.

Status Byte	Data Byte 1	Data byte 2
(1001 0100)	(0100 000)	(0101 1001)

Figure 3: Table illustrating how the three eight-bit types for MIDI information.

Status Byte - Starts with a 1, and carries information on 'Note-on' and 'Channel'. Data byte 1 - Starts with a 0. This carries information on the note played. Date byte 2 - Starts with a 0, and carries information on the note's velocity. Messages that are received by and sent by MIDI fall into two MAIN channel categories: Channel and System messages.

CHANNEL MESSAGES

Channel messages are made up of two different message types – Voice and Mode messages. Voice messages carry information on the channel's messages: Note-Off; Note-On; Polyphonic Key pressure; Control Change; Program change; Channel Change/After Touch and Pitch-bend Change. These are the most relevant message types most receiving interactive system like this use. Mode Messages are made up of 4 modes: OMNI, MONO/POLY, ALL NOTES OFF and LOCAL CONTROL.

SYSTEM MESSAGES

System messages communicate information specific messages, but only to specific devices. They are made up of three message types: common; real-time and exclusive. Common Messages are messages intended for ALL devices in the system. Midi messages are: MIDI Time Code; Song Position Pointer; Song Select; Tune-Request and End-of-Exclusive. Real-Time messages are intended alsp for ALL devices but contain only status bytes and are commonly used for synchronization. Messages are: Timing Clock - pulses per quarter note (also called 'ticks') 24, 128, 240, 480, 960; Start and Stop (control a sequencer); Active Sensing (sends a MIDI signal every 300ms); Reset.

IDENTIFYING INTEGRATION

The most useful information prone to constant change was Channel messages – messages that detect change from keys using *note-on, note-off, note-number* and *note-velocity*. Many digital-audio-workstations such as *Ableton Live, Logic Pro* and *Max* receive and send this MIDI information. For instance, an example of MIDI being received by an application, when playing the note[C] for half a second on the piano would be: [90 36 106 80 36 47].

[10] Software that integrates audio synthesis in a digital audio workstation environment. It uses digital signal processing to simulate traditional recording studio hardware in software.

Data	Definition
90	Note-on
36	Note-number
106	Note-velocity
80	Note-off
36	Note-number
47	Note velocity

Figure 4: The first three messages in the table above are received as soon as the key has been pressed; the remaining three are received as soon the key has been released.

It was important to quickly find a technology that would support cross-integration of both MIDI and graphics throughout the subsequent phases of the project; the solution is discussed in the design and implementation chapters of this report. First however, it is important that the domain knowledge surrounding computer-graphics is also founded, so that requirements specification can be illustrated clearly.

3.5 COMPUTER-GRAPHICS STANDARDS

OPENGL

OpenGL is one of the largest graphics standards by Khronos.org, existing as an API to support a variety of data types and functions that render 2D and 3D graphics. Other notable graphics standards include WebGL, intended as a JavaScript API compatible with web browsers, and Khronos.org's more niche OpenCL framework, which executes across heterogeneous platforms. Although OpenGL's function definitions are similar to the programming language C, they are language-independent, and many of the bindings use and are accessible to be used by other technologies that include WebGL, iOS and Android. An overview of the major operations that an OpenGL application would perform to render an image would be:

- Specifying geometric construction of primitives;
- Executing shaders to perform calculations on a primitive's vertices and fragments;
- Rasterization;
- Fragment shader execution for final colour and position;
- Additional per-fragment operations.

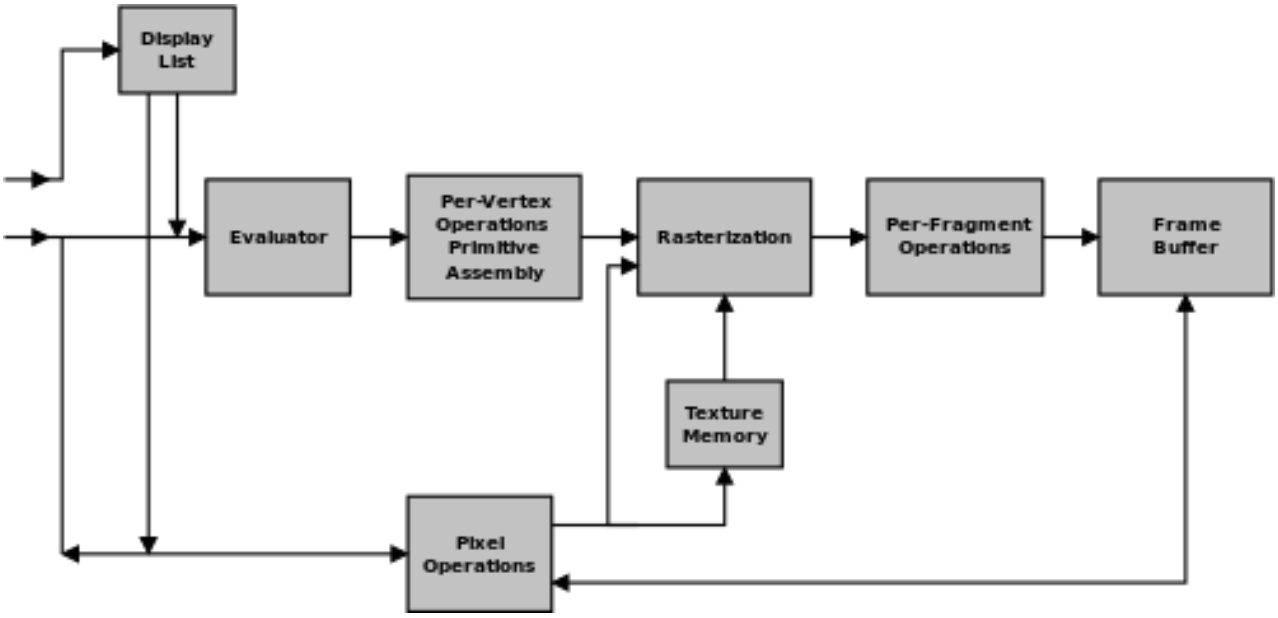


Figure 5: An illustration of the graphics pipeline process.

As illustrated above, the graphics pipeline process ends at the frame buffer, in which most we most commonly derive the frame-rate, or refresh-rate of the buffer per second. Many of the major operations that occur within the graphics pipeline process involve the shading language, a user-defined program designed to run the graphics processor.

OPENGL SHADING LANGUAGE (GLSL)

The OpenGL Shading Language (GLSL) is used to write user-defined shaders in OpenGL to run on the graphics card (GPU), and modify the appearance of 3D objects creating light effects for virtual scenes, processing images and displacing vertices. Its nature is to execute one of the programmable stages of the rendering pipeline; the vertex and fragment units. Therefore, an OpenGL GLSL single shader is comprised of two programs: the vertex shader and the fragment shader.

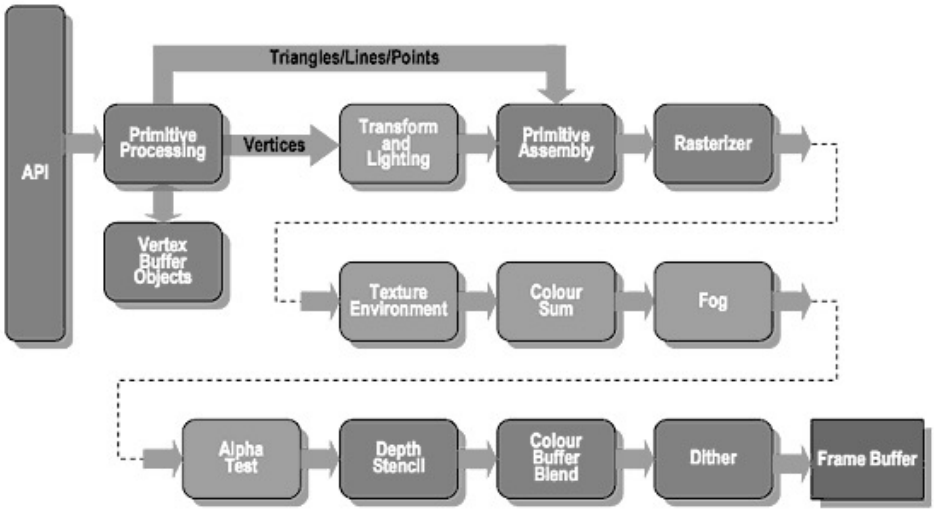


Figure 6: A flow-diagram representing the fixed-functional pipeline. The Vertex and Fragment portions are programmable units that can alter the pipeline using shaders.

Any of the system's 3D geometry is rendered by both vertex and fragment processors. The benefit of utilising shaders is that operating any matrix data sets have the potential to significantly increase performance efficiency when given instead to the GPU. Graphics cards have up to 128 vertex pipelines

and 176 fragment pipelines that can each operate in parallel, enabling a much higher volume of image processing effects at HD resolution with full frame rate. It would seem that GPUs will continue to increase in performance at faster rates than the CPU, in accordance with Moore's Law of performance doubling every 18 months. Shaders are often written in using the GLSL-style syntax, which is heavily built upon C++.

Parameters outside of the shader can be declared as uniform variables, acting as a means of altering certain GLSL operations within both the fragment and vertex programs the shader is made up of. With this, a host application such as Max programs is able to affect change upon this parameter through successful 'binding' declarations. As such, the visual-interactive system in question should have a suitable selection of parameters that are externally and internally modifiable for user-interface interaction with the system.

3.6 REQUIREMENTS SPECIFICATION

REQUIREMENTS PRACTICALITIES

Based on the identification of the project's stakeholders, and the preceding domain knowledge outlining necessary standards of the project, it was now possible to both define and classify the requirements into *functional and non-functional.* The specification is broken into 3 phases, to illustrate how requirements were derived iteratively, with design and implementation on going throughout each phase. The requirements are validated in the Evaluation Chapter.

It should be acknowledged that outside the scope of this project, it would be more effective that requirements be further/sub-categorised into *structural* and *performance* requirements, but this was not necessary because of the quantity of requirements listed. Note that many more requirements were initially identified that fell outside the project's scope, or were modified and/or remove

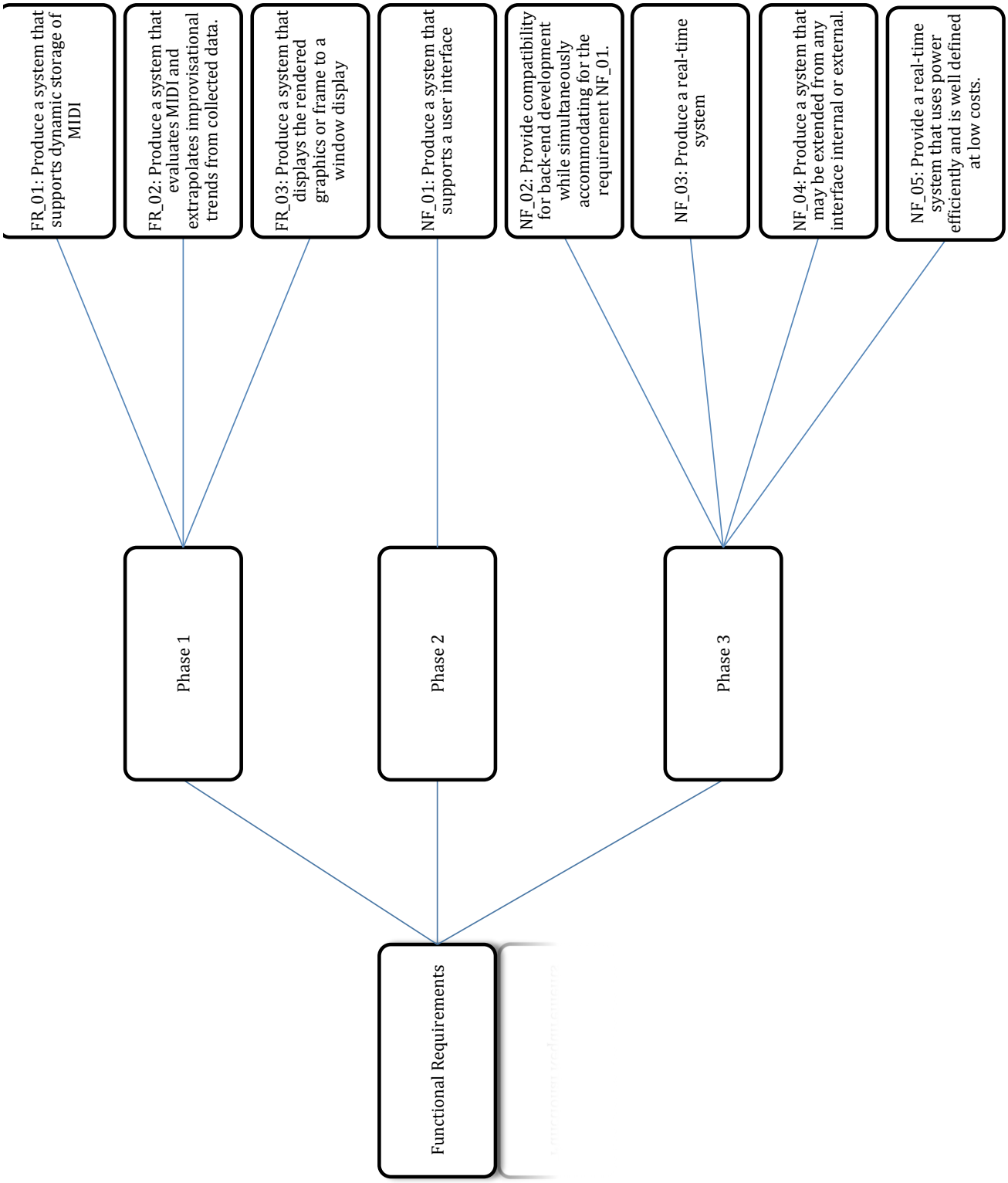


Figure 7: The full requirements specification derived over three iterative phases of the project.

3.7 SUMMARY

Following a specification, the requirements listed above will often be referred to by their ID tag (FR_##) for the remainder of this report. The following chapter describes the design process of this project, and gives a design context and principles that lead to the design process itself.

4 DESIGN

4.1 INTRODUCTION

This chapter uses the defined requirements to present the design. This includes the overall design process and resulting key issues, and provides relevant Max UML at architectural and behavioural levels of the system. This chapter also provides a table of all the modules in the system.

4.2 DESIGN PRINCIPLES

CHOOSING A DESIGN TOOL

From the beginning of the Software Design Life-Cycle (SDLC), Max was actually chosen as a UML-type tool intended to visualize the design of the system. Max is not an official UML tool, but it shares similarities to UML tools such as *StarUML* or *Visual Paradigm.* As well as being a visual-programming language it was also, as discussed below, the chosen tool for the system's implementation.

A REAL-TIME ENVIRONMENT

A 'real-time' system in computer science describes hardware and software systems subject to a 'real-time' constraint. In this context, the term is not to be confused with real-time operating systems, but there are similar qualities. In the context of designing this system, 'real-time' takes on an important notion: *responding to triggered events from a source (MIDI) within the global tempo set for a piece of music*. This is dictated using beats-per-minute (BPM), and is often utilized within software and hardware recording environments. This means that a midi-keyboard communicates to a system that realizes events with a global time constraint, and this system reacts to this processing. This could be an interactive system constrained by recurring events that result in the production of a series of vertex or fragment transformations.

Fortunately, Max was built from the ground-up to accommodate real-time environments without too much design and implementation maintenance. However, for an interactive system such as this, what was important to be aware of the two distinct event priority levels that exist within Max, and any additional threading issues that may arise when any important design considerations were being made.

EVENT PRIORITY IN MAX

Max events are categorised into two levels: low-priority and high-priority. Because events in Max cause a sequence of messages to be sent from one object to another object through a Max patcher[11] 'network', the way in which these messages traverse a max patcher is at 'depth' first, meaning that any given path in the network is always followed to its terminal node before messages will be sent down adjacent paths.

One of the key design considerations to take into account is that MIDI in Max has timing information associated with each event; allocated a scheduled time for when they are to be executed. Any event that reads or writes MIDI falls into the category of high priority, also known as 'scheduler-events', whilst other events (a mouse-click or keyboard-key press) fall into the low-priority or queue event category. By default, Max runs both priorities on the same thread, but with Max's 'overdrive' feature switched on,

[11] Facilitates the modularization of patches within patches.

runs two threads for the execution of events causing high priority events to be able to interrupt low priority events.

This means that any messages being passed through the patcher will have threads that work simultaneously. This is a clear advantage especially when designing a system to operate on multi-processing computers - a processor for each priority. On the basis of this, it is not necessary to overhaul the architectural or behavioural design as Max does this for you, but it is fundamentally important to factor into design for larger interactive systems, especially if end-users exploit concurrency.

DESIGN PRINCIPLES I: MODULARIZATION BY ABSTRACTION

Using techniques of prototyping with relevant technologies and observing the way other similar software products worked as mentioned in the requirements chapter, it became clear that to meet these requirements simultaneously would need to involve a break-down of the system into discrete and independent modules, to be capable of carrying out tasks independently.

This 'divide and conquer' approach is meant that smaller components were easier to maintain, and could be divided based on their functional aspects. It also meant that depending the technology chosen to deploy the system that components with high-cohesion could be re-used again. The disadvantage of such an approach meant that there could be limitless possibility of interconnecting modules, making the system more difficult to test performance and reliability in the long-term.

Modularisation was achieved by implementing abstractions. Abstractions in Max are essentially sub-patchers that are saved as external files, and can be used just like a standard Max object, without being able to view the back-end. Directly loading objects into patch windows to provide a more efficient working environment can create much larger interactive modular systems that are easier to maintain.

DESIGN PRINCIPLES II: PRACTICAL CONSIDERATIONS

Often, as a design principle, the separation of interface and implementation is considered when deploying and integrating the front and back-end of a system. However, to satisfy the requirement of **NF_02**, it was about creating a hybrid of the two. The user-interface had to exist within the development environment (Max), but there still had to be a noticeable separation of the user-interface and the back-end of the system. This was to hide any unnecessary low-level design behind the front-end of each module.

SUMMARY

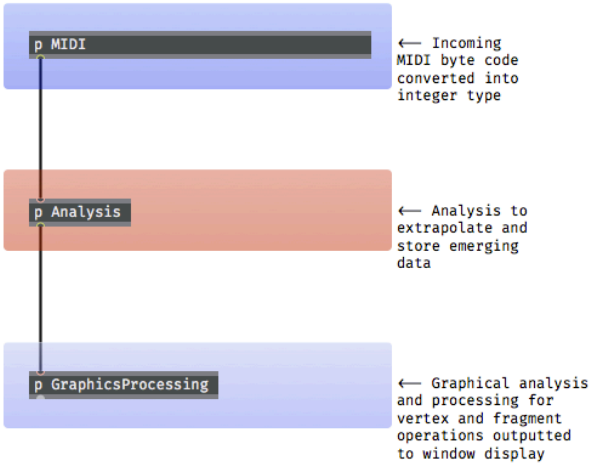
These design considerations above formed part of the design process: the system's architectural design and module interaction behaviours. Over the course of the project, this series of modules were built into a Max package, and this package was called '*Kluster*'.

4.3 THE DESIGN OF THE SYSTEM

ARCHITECTURAL DESIGN

The architectural design formed throughout each phase of the SDLC based on the approach of modularization in a real-time environment attempted to facilitate the overall characteristics of the software requirements for *Kluster*.

The initial architectural design was centered on the idea of having a procedural pipeline that started from its source, the MIDI input, and ended at the Window display. Everything in-between would be a module utilizing a variety of functionalities and passing their output to the next module's input.



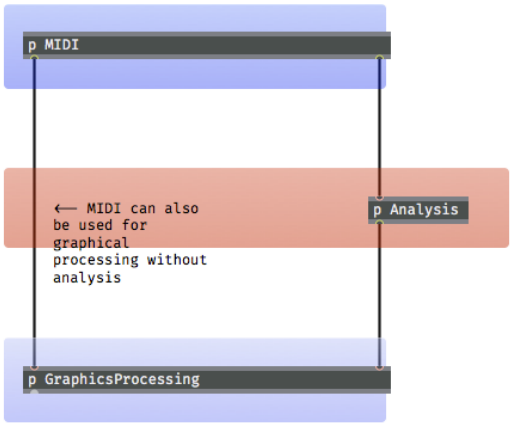


Figure 8 (above left): The initial architectural design of the system

Figure 9 (above right): The final architectural design after re-evaluation. Whilst the source (the MIDI input) and the final module (the Window display) remained the same, it became clear that the output of certain modules could be passed to the input of modules much further down the pipeline. This gave the possibility of more flexibility, but resulted in a more complex system to maintain.

MODULE INTEGRATION

It is important to note that this just one method for demonstrating how the modules can be integrated with one another. Depending on the output and input of each module, there are limitless ways in which the modules can be used with each other in the development environment and with other Max internal and external objects. The result was a more flexible system, at the cost of being more difficult to test

Module Name	Module Description	Boundary	Input(s)	Output(s)
Midi-in	Receive Midi in	MIDI	MIDI-in	Number<Integer>[0-127] Number<Integer>[0-127] Channel Number<Integer>[0-127]
M-Filter	Filters volume of Midi data based on probability.	Analysis	Number <Integer> [0-127] (2x)	Number <Integer> [0-127] (2x)
Nodec	Evaluates a set of functions using node-interpolation	Analysis	Number <Integer> [0-127] (2x)	Message <List> (8x)
N-Filter	Filters volume of Node-message data based on probability passed-thru.	Analysis	Message <List> (4x)	Message <List> (4x)
N-Unpacker	Unpacks Node messages containing evaluated functions and outputs them individually	Analysis	Node <List>	Number <Float> (8x)
Z-Plane	Evaluates a set of functions using graph filter poles and zeros in the Z-plane.	Analysis	Number <Integer> [0-127] (4x) TBC: Still in design phase	Message <List> (4x)
Z-Filter	Filters volume of Z-Plane-message data based on probability.	Analysis	Message <List> (4x) TBC: Still in design phase	Message <List> (4x)
Z-Unpacker	Unpacks Z-plane messages containing evaluated functions and outputs them individually	Analysis	Message <List> (4x) TBC: Still in design phase	TBC
Matric	Receives inputs and fills 1-4-plane matrices and evaluates them	Graphical Processing	TBC: Still in design phase	Mat, Mat2, Mat3, Mat4
Particulator	A particle system	Graphical Processing	TBC: Still in design phase	None
FpWorld	A first-person camera module giving a World view	Graphical Processing	Position [x][y][z]; Lookat [x][y][z]; Rotate[x][y[z]	None

Figure 10: All the modules that exist within the system (The Kluster Package for Max). Highlighted rows indicate that are discussed in the subsequent implementation chapter, relevant to the requirements being addressed. Some TBC on modules indicate that the modules are still in design/implementation phase, and their inputs are not yet confirmed.

MODULE BEHAVIOUR DIAGRAM

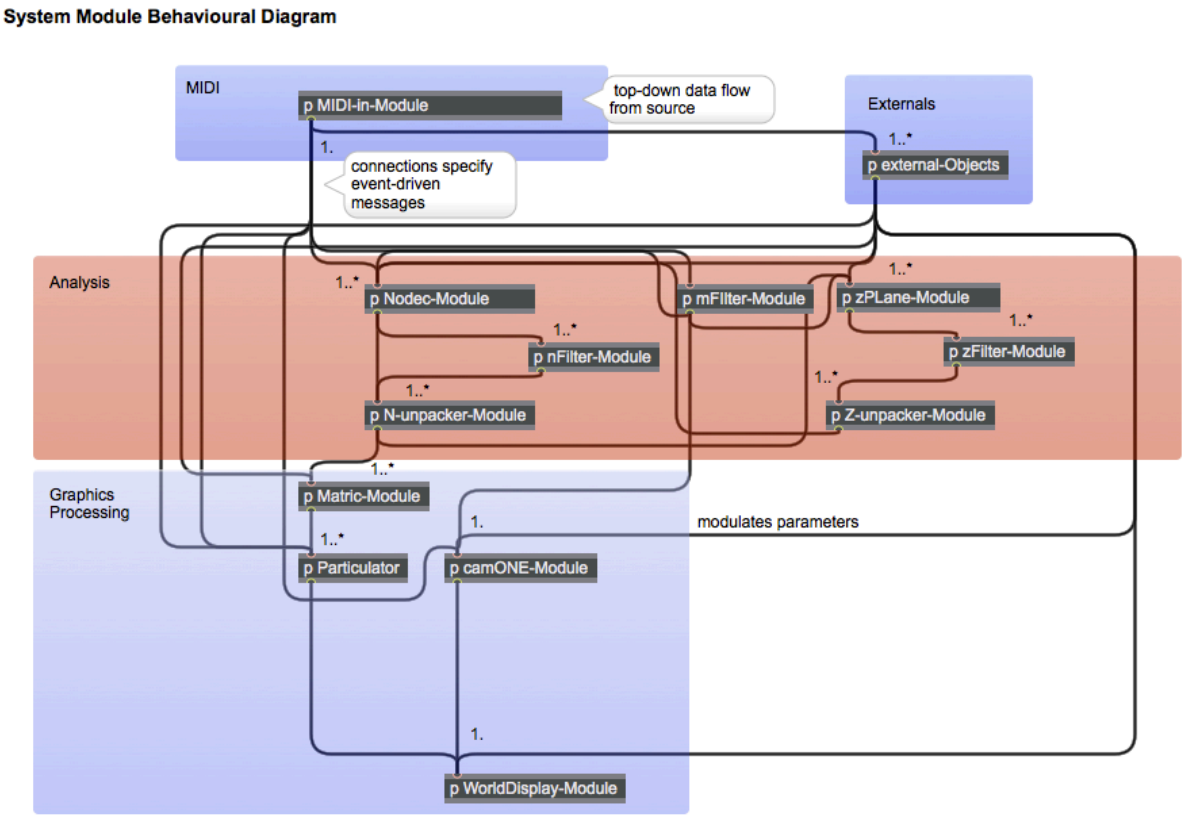


Figure 11: Max UML for the system module behaviours that lays out the boundaries from the architectural design, and each module, specifying interaction and key multiplicities with one another.

4.4 KEY ISSUES IN THE DESIGN

DATA HANDLING

The Nodec and Z-plane modules were designed from the outset to satisfy NF_01, and other OpenGL-based modules like Matric or Particulator were designed to retrieve and use this data remotely, without having to be patched together. This is a useful feature of the [coll] object, which stores data in the form of a .txt file.

Determining the frequency of data storage and retrieval was a design would have an impact on certain information being read in other subsequent modules. The key issue in design was not just about how the system should store data as in Max this is relatively is straight forward; it was about when. Nevertheless, the design decision in this process was for the *Nodec* module to store not only MIDI data, but any emerging data it, as this was a module performing real-time analysis.

DISTRIBUTION OF COMPONENTS

This distribution of components is not only necessarily concerned with how the system being engineered was modularized by abstraction, but how the system also interacts with certain aspects of hardware. They were both considered interchangeably throughout the design process.

Depending on which module was chosen by the end-user to be deployed in the Max environment, modules would compute on the central processing unit and/or the graphical processing unit, meaning that there were aspects of heterogeneous computing that existed within the system. One of the key design considerations was to exploit these two hardware components as much as possible within the system, and to divide portions of the modules where necessary as middleware based on the data that hardware was computing and for what reason. Modules such as *FpWorld* had objects that primarily existed in Max (Jitter portion [jit.gl]) at the back-end, and certain objects such as [jit.gl.slab], [jit.gl.pix] and [gen] were low-level objects that computed matrix operations on the GPU for vertex or fragment operations in OpenGL.

4.5 USER INTERFACE DESIGN

There are two alternate views when designing a module in Max: The back-end, and front-end (its user interface). When a module is deployed in Max, the module's graphical user interface, formally known as 'Presentation mode', and back-end are wrapped in a sub-patcher ([bpatcher]). This module can then be embedded within another patch, with only its user-interface present to the end-user. This is abstraction. The benefit of abstraction is that module hides much of the complex back-end that is unnecessary at higher-level tasks. In addition, every module was designed to utilise the [thispatcher] object, which enables users to specify the number of inlet and outlets that come in and out of the embedded patch (module).

Because the back-end of each module was kept separate from the user, it was important to appropriately select what information should be visible on the user interface, and what inputs and outputs should be present. This was not usually a decision made when designing the user interface itself, but more a decision made when designing each module's back-end. The design of user-interface was influenced by some of Max's module packages, including Max For Live (M4L), VIZZIE and BEAP.

4.6 SUMMARY

Testing by simulating performance, perhaps with different computers or spaces, and by continuous prototyping, to evolve each module based on a number of issues that may arise, can analyse much of the design of the system considered in this chapter. Whilst the next chapter discusses implementation issues, it is important to note that the two chapters were not entirely separate in the design process; there was constant analysis and revalidation of the system's design that occurred throughout the implementation process too which ultimately had an impact on forming an iterative SDLC in the first place.

5 IMPLEMENTATION

5.1 INTRODUCTION

This chapter gives an overview gives as overview of Max for the benefit of the reader's understanding of the module implementation. It then goes on to discuss three modules in detail, highlighted in the table in 4.3.3 of the *Design* Chapter. Each module includes a description of its features, practical considerations and how it was constructed and tested.

5.2 CONSTRUCTING WITH MAX

This section explains some of the key syntax in order to understand the construction behind each module. A full set of in-depth tutorials can be found on Cycling74.com, and in the reference of the Max software itself.

MAX SYNTAX

Most of Max's routines exist as shared libraries, and its API allows third-party development of new routines or 'external objects'. The framework of Max is split into three components: Max, MSP and Jitter. In addition, programming in Max can be better undertaken by writing "externs" in C or C++, by embedding Java and JavaScript files or by importing entire interpreters.

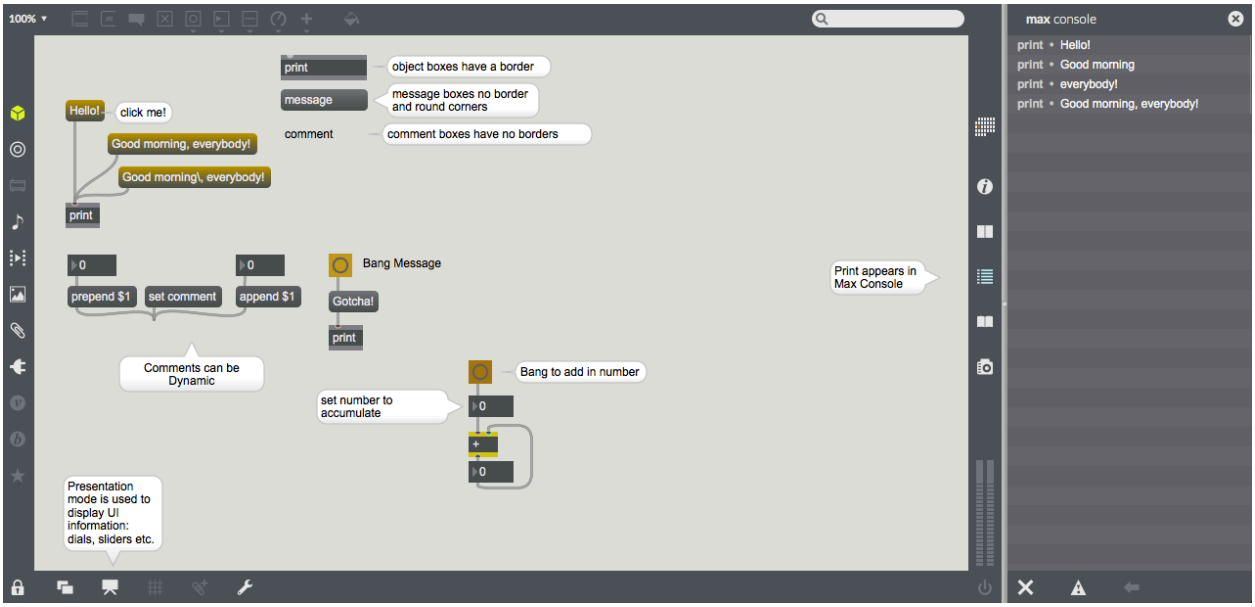


Figure 12: Max's development environment window (locked) illustrates some of the basic objects and messages. Right: Max's console

As discussed in the Design Chapter, the design pattern surrounding this *was event-driven architecture* designed atop *message-driven architectures*, where the instance of an event, sent through patch chords, causes significant change in the state of the software. The design and implementation of the software meant that there would be a constant transmission of events among these loosely coupled software components, defined in the last chapter as '*modules*'. Within each module, there was a desire to minimize the modules complexity in terms of both the UI and each module's back-end.

In Max, this can be implemented through Max's 'Presentation mode', where certain objects such as sliders or dials are visible in the patch. When a module abstraction is loaded into another patch, I constructed the module so that its presentation mode is enabled, hiding all back-end development from

view. Because of the iterative SDLC model, there was often construction for both validation and verification purposes, especially when it came to minimizing functional complexities for certain modules like Nodec. In addition, modules were implemented simultaneously, some taking longer than others to prototype.

5.3 MODULE DESCRIPTIONS

In the following sections, the *M-Filter, Nodec* and *FpWorld* modules will be described sequentially. A module description will be given, its construction process, its integration and user interface descriptions and any notable practical considerations that were made. Not all modules are reported, but a list of other modules can be found in the *Design* Chapter, some of which are still in the engineering phase.

5.4 THE [KL.MFILTER] MODULE

MODULE DESCRIPTION

MFilter is a simple module that uses probability to control the flow of MIDI data through a two-gate. Depending on implementation, this can be beneficial for CPU performance, aiming to satisfy the requirements of **NF_05**. There can be multiple copies of the module, working in parallel with multiple MIDI channels and inputs.

Figure 13: The M-Filter or Midi-Filter module at front-end, hiding unecessary of its complexities at back-end. The three half-moon objects at the top of the module indicate patch inputs to receive inputs. The bottom represent outputted Midi-data.

MANAGING CONSTRUCTION AND TESTING

The module has a total of three inputs: The note velocity, the note value and the % of probability that the message will pass through to the module's output. There are two outputs: the note velocity and the note value. Two integer inputs feed into the module's input inlets, through a two-gate and out two outputs. This probability is presented as a percentage (%), and can be modulated as a parameter externally, or manually by using the user-interface dial. If a random integer generated between 0 and 100 is greater than or equal to its comparator (the % parameter displayed on the UI), this will use a gate to trigger the module's two-gate to open and allow both inputs to pass through to module.

The module was built and tested from the back-end, and throughout the design life cycle, it was eventually deployed with subsequent modules, such as *Nodec.* To test the system, the module was fed numbers at random time-intervals to assess the performance and assert the correct output.

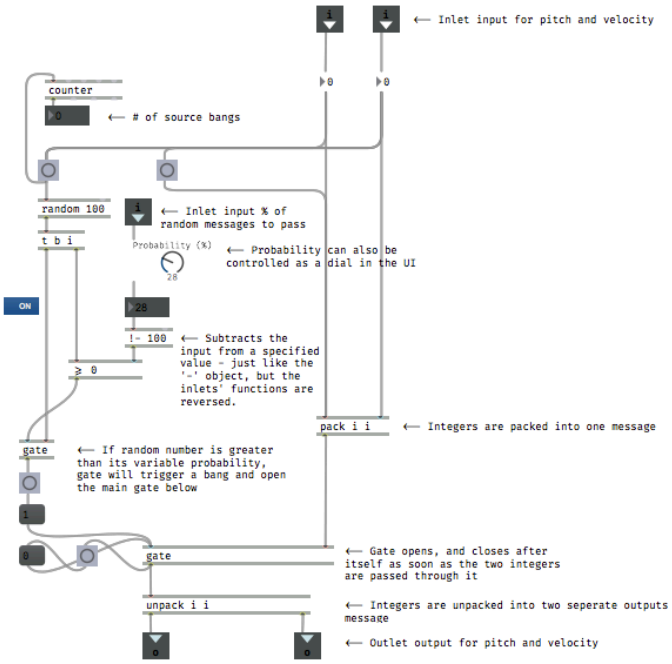


Figure 14 (Left): The source code was organised procedurally, and constructed from its input (top [i] of the patch), to the output (bottom [o] of the patch)

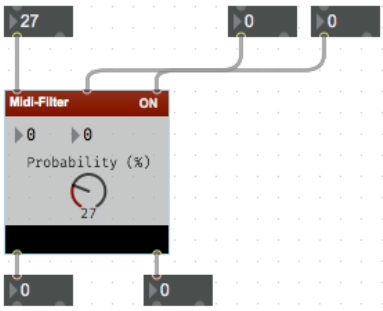


Figure 15 (Above): The user-interface for the Midi-Filter (M-filter) module, integrating with internal Max objects

PRACTICAL CONSIDERATIONS

In this module, the velocity and note value ranging from 0-127 are treated as separate inputs. In MIDI, you cannot have a note value without a velocity value. Therefore it was important that the module's back-end treat the two inputs as paired-values, which is why the object [pack] (an object that concatenates numbers into a list) was implemented before it reaches its gate, and [unpack] (the opposite of [pack]), to separate the messages before they reach the output were used.

USAGE, INTEGRATION AND USER INTERFACE

The module has one dial controlling the probability of a message pass-thru. The module can also be turned off, giving no output data at all. As illustrated in the design chapter, the module aims to be deployed between any proceeding MIDI module and any its output received by analysis modules, such as *Nodec* or *Z-plane*.

5.5 THE [KL.NODEC] MODULE

MODULE DESCRIPTION

Nodec is a module that uses node interpolation to evaluate a set of functions. This module aims to provide extensible functionalities: to perform functions based on Node data involving predictive analytics and machine learning tasks, to store data and output node information as a list for other subsequent modules to use. It does this by identifying relationships between a single instance (a MIDI note or event) and a set of nodes that lie proximity to a given point. As well as providing two-integer MIDI for this, the module also provides inputs to modulate various parameters from an external source.

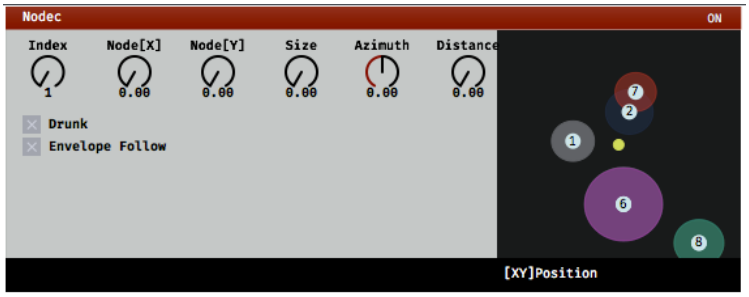


Figure 16: The Nodec module, with 8 inputs highlighted and an array of selected user-interface dials for manual interaction too. The right shows a graph of coloured nodes that evaluate a set of functions based on where the mapped single MIDI instance is (yellow point) . The user may select the index using the dial and alter the node's diameter or move it around. Each dial can be modified by an external. The module outputs a list of evaluated functions from every MIDI instance.

MANAGING CONSTRUCTION AND TESTING

This module aimed to satisfy the functional requirements **FR_01 and FR_02**. The source code was organised in order of functionality, and designed to encapsulate sub-patches that carry out a specific function. The module takes two integer inputs, MIDI velocity and note value ranging from 0-127, and outputs a message containing a list of values from returned functions from each specified Node 1-8.

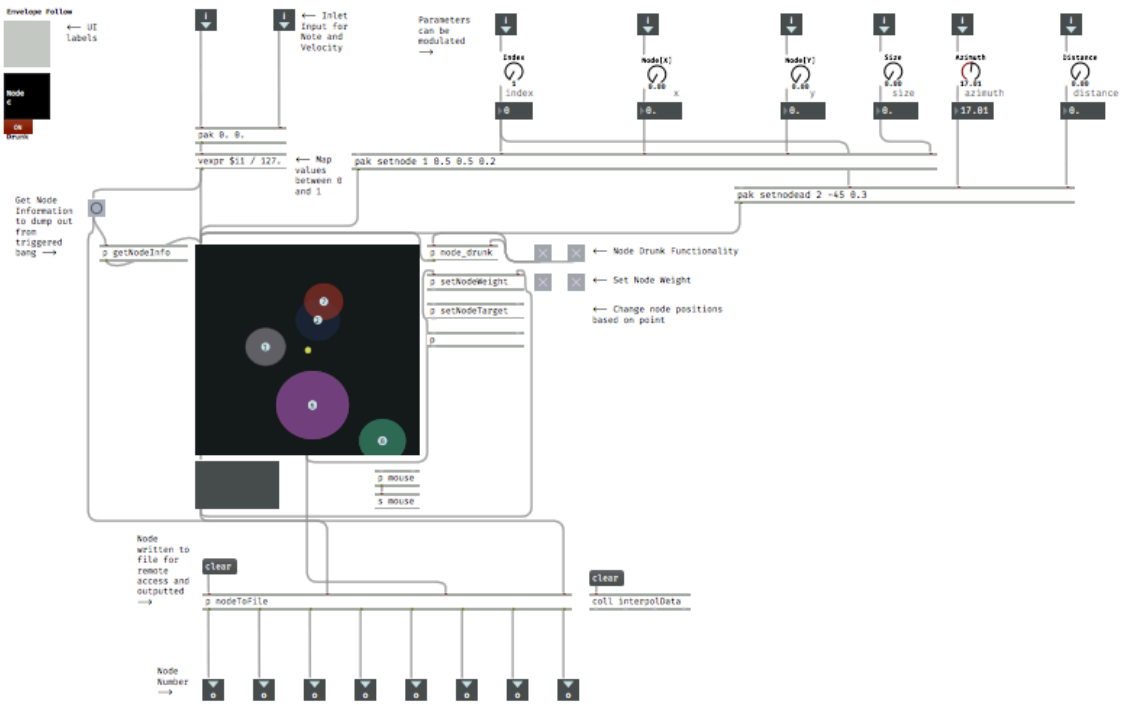


Figure 17: The back-end of Nodec. Sub-patches starting with 'p' are encapsulated functions. Some of its features are not yet fully implemented.

POLYNOMIAL INTERPOLATION USING NODES

When inputs are received, Nodec will normalize its values between floating point values of 0 and 1, and map this to a 2-dimensional plane to give an (x, y) coordinate. More data can be extrapolated if there are more points lying in proximity to a point; referred to as Nodes. This means that with 8 nodes present, every point dumps out 8 separate pieces of data. With this new data, further functionality was then implemented.

NODEC FUNCTIONS

After polynomial interpolation, Nodec was engineered to evaluate a set of functions relevant to **FR_02.** The module hoped to encapsulate functions that re-enact attributes of improvisatory performance set out in 3.3 of *Requirements Chapter*: to *listen, react, augment*[12] and *create*.

Node Weight: This function would *react* and conditionally alter a Node's weight, by introducing a conditional operation that can increase the diameter of a node depending on the respective point's proximity to the node's centre.

Node Movement: The way nodes move is something that can be explored to facilitate more change in data. Having nodes predict where the next point might be by listening and reacting to each point begins to identify where improvisational trends may lie. Implementing simple reinforcement learning tasks in machine learning could do this, and each node would become more aware over the course of time, increasing its overall awareness of a point over time. Gaussian Mixture Models (found in the ml.lib library for Max [ml.gmm]) and/or k-means clustering-based tasks could have been useful here, but its implementation fell out of scope of the project in terms of the timeframe due and a lack of documentation for the [ml.lib] package for machine learning tasks. This was recognised for future consideration.

Drunken Nodes & Followers: This function aimed to *augment* aspects of each node's position, by introducing functions that randomize all aspects of node movement and growth, and nodes that follow one another. Implementing randomly triggered bangs that outputted a range of random numbers could control the (x, y) position, weight and randomly select which node was being controlled.

STORAGE AND OUTPUT

To satisfy the requirement of **FUN_01**, triggering a 'get' message to dump out information when the module received a message meant that these evaluated functions could be pipelined to a single file destination. This file could be remotely accessed by other modules; a feature Max's [coll] object inside Max, which stores the evaluated functions inside a *.txt* file. Many of the stored numbers range from 0. – 1, which, discussed later, is a useful range for modulating or accessing OpenGL parameters.

As opposed to accessing the .txt file ([coll] object) remotely, the module also outputs a set of evaluated functions as a list. Each output is a Node-message, ordered from left-to-right. If required, the developer can unpack and scale any given output node list using the [unpack] and [scale] object provided by the Max's internal API.

[12] The process of adding a sound a fragment, or imitating other performers.

PRACTICAL CONSIDERATIONS

The separation of functionalities was a key consideration when implementing this module, especially in circumstances where source code was being repeated. Similarly, complex algorithms in Max are far more difficult to maintain than in text-based procedural or object-orientated languages. A lack of clear encapsulation techniques that exist within more object-orientated design would have been more effective when implementing more complex functions for this module - getters and setters alike. Max is actually able to embed JavaScript files into its patches, so producing separate JavaScript files may have been more effective in engineering object-orientated techniques using JSON.

INTEGRATION AND USER INTERFACE

The user-interface of the module gives the user a set of dials, some of which can be modulated by an external source. It also provides a 2D graph, displaying the point and each nodes position. The module can be turned off, giving no output data at all. The module is designed to integrate with the *N-unpacker* module, which unpacks each Node's list respectively, or can be used with any other relevant Max API objects. The variety of data this module provides makes it useful to integrate with any module or object that requires a large set of data in real-time.

5.6 THE [KL.FPWORLD] MODULE

MODULE DESCRIPTION

When loaded into Max, First-Person World Module (FpWorld) or [kl.fpworld] is module is intended for displaying a world-view or the viewing frustum on a separate window display and providing a set of user-interface controls intended to drive the camera's position around the world. This module aimed to satisfy the requirements of **FR_03**.

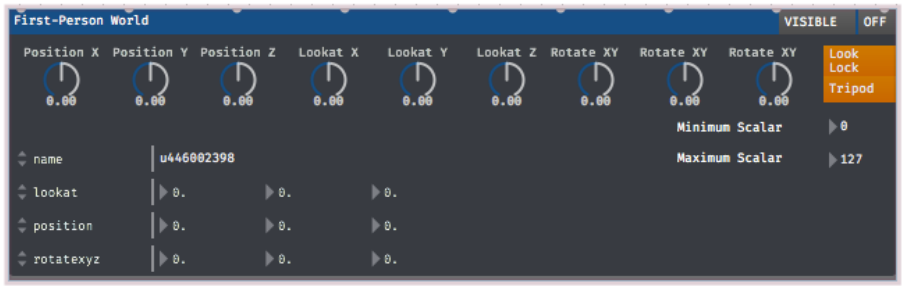


Figure 18: FpWorld user-interface, hiding the complexities shown in Figure 19. Hotkeys enable the window to become visible, and for frame-rendering to be executed or frozen. This is displayed in the top-right of the module. The camera movement can be affected with tripod and look-lock modes.

MANAGING CONSTRUCTION AND TESTING

The module was based upon three of Max's internal Jitter-based objects – [jit.world], an object capable of displaying a window, and [jit.gl.camera], an OpenGL first-person camera, and [jit.anim.drive] a Max object based on driving objects in a 3D space. No connections are needed as inputs in order to activate the system – the end-user may just load it into the Max environment and use it instantly.

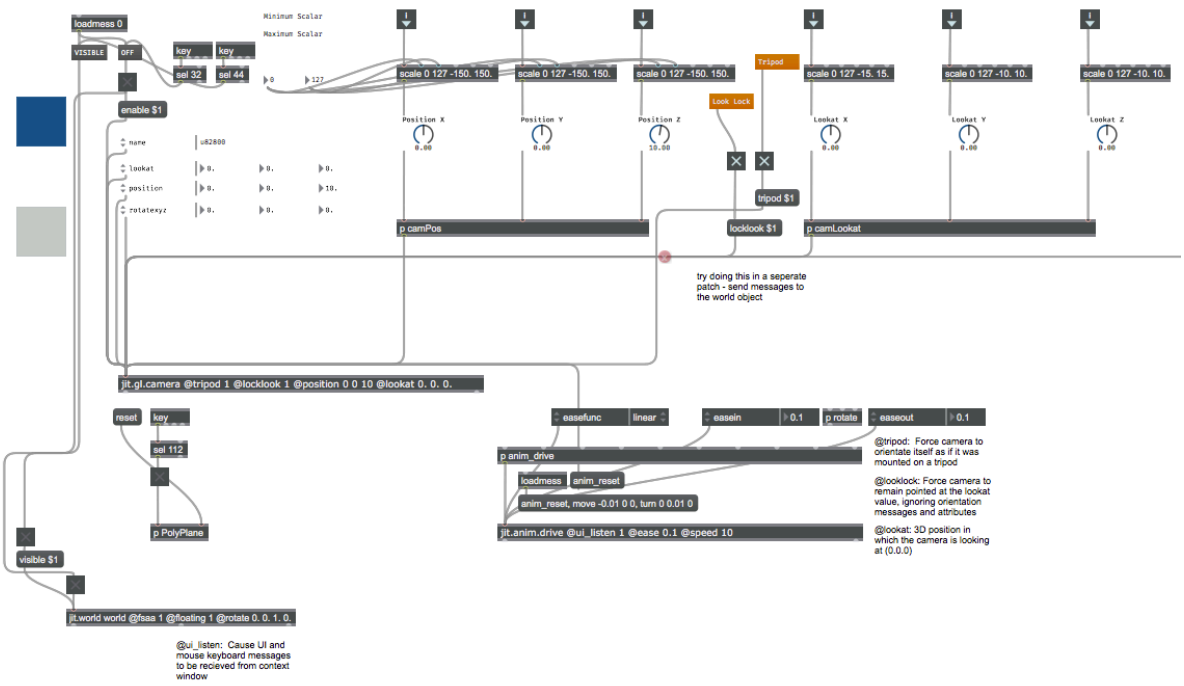


Figure 20: An overview of the back-end of the FpWorld module. Note that not all of the back-end is visible here. Observe how the encapsulated patches (with p) are encapsulated functions that send messages to each coordinate point of the [jit.gl.camera] object, which can also be controlled externally. Inputs are by default scaled from 0-127 to facilitate MIDI as an input. These scaled parameters can be changed if desired.

A white-box approach was taken in testing the module. Performance testing was done through alterations to the user-interface dials. The display window can be closed and opened again through hotkey mapping at the back-end, and rendering may be halted using the spacebar, which freezes the current frame.

PRACTICAL CONSIDERATIONS

One of the biggest challenges was driving a camera with the object [jit.anim.drive], which caused significant performance issues that affected **NF_05**. The alternative to this is by using the [gen] object instead, which utilises the sin and cosine functions for camera rotations. This is for future considerations.

INTEGRATION AND USER-INTERFACE

FpWorld aimed to be integrated with any modules for this existing system. The module also aimed to accommodate the requirement of **NF_04**, so that it can be extended from any interface and external – Max package VIZZIE's INTERPOL8R module was used to test whether it can interpolate between values so that it could affect the camera position *rotation*, *look-at* and *position* (x, y, z) parameters.

In addition, the module's parameters controlling the camera were scaled so that it may could be directly manipulated by MIDI itself, rather than having to be controlled by modules that perform data analysis. This is a feature of the system's architectural design outlined in 4.3.1 of the *Design* Chapter.

5.7 SUMMARY

This Chapter has discussed the implementation considerations of three modules within different boundaries of the system, in relation to the requirements and design considerations. The following chapter reflects on the engineering process by revisiting the requirements again, validating their existence and verifying their presence and/or effectiveness in the system.

6 EVALUATION

6.1 INTRODUCTION

This chapter evaluates the system through requirements verification and validation, and where possible, comparison with other interactive systems or products is made. The chapter concludes with an evaluation of the system's design.

6.2 REQUIREMENTS VALIDATION AND VERIFICATION

In the *Literature Review* Chapter, section 2.7 outlined the software potential using MIDI as opposed to audio as the potential source of analysis for this system. **FR_01** was derived to support this potential, so that MIDI information could be accessed within an OpenGL context. An alternative requirement could have just been to not store this information at all, at the cost of tracking historical patterns in MIDI for more complex functionality in other parts of the system. Within the system itself, the Nodec Module stores these MIDI events sequentially, along side other evaluated functions that the module performs, resulting in a each index having a MIDI note, along with a list of evaluated functions. Other modules can also access this list remotely, without having to be connected through patch chords. However, neither Nodec nor any other module could modify or normalise evaluated data, resulting in data redundancy. A more focused derivative could have been to design specific behaviour of this data storage by illustrating relevant database UML - a data model, or an entity-relationship model perhaps.

Evaluating and extrapolating improvisational trends from MIDI was a requirement that stemmed from the potential of data analysis in 2.7 of *the Literature Review* Chapter. **FR_02** was an ambitious requirement that inadvertently formed the basis to perform all other functional and non-functional requirements defined for the system. Because MIDI is in the integer data type range 0 -127, there is not much compatibility potential towards direct OpenGL functionality. This in itself was a more viable argument for using audio for data analysis instead of MIDI, but there is already a great deal of contributions to using audio in visual-interactive systems. In an attempt to verify this requirement, it was split into two separate concerns: evaluating MIDI, and extrapolating improvisational trends.

The operation around the evaluation of MIDI took place in the *Nodec* Module. This involved normalizing MIDI so that it could be graphically interpolated using nodes. Extrapolation of improvisational trends was also primarily achieved in *Nodec* too. Functionalities included *node growth*, *drunken nodes* and *node following* to achieve a more diverse set of extrapolated data. A separation of functionality meant that the module's design had the potential to have a high cohesion, but this was difficult to evaluate without more functionality present. There could have been more clearly defined functions involving predictive analytics and machine learning tasks that relate to the four attributes in improvisation as defined in the requirements chapter, and a more extensive user interface to go with it. This is all for future consideration.

For a visual-interactive system, it is evident that there be some form of window display. This formed the simple, but necessary requirement **FR_03**. The FpWorld (First-Person World) module attempted to encapsulate the necessary functionality and user-interface controls necessary for a visual-interactive system, by providing series of dials that could drive the camera around the world through a first-person camera view. This requirement partially derived from 2.5 in the *Literature Review* Chapter, which discusses the need abstraction for user-interface design. Other internal and external Max objects are capable of displaying the world window-view too, such as [jit.world] or [jit.window], but these are more low-level internal objects, and have no UI or module with appropriate camera controls included in them.

Similarly, the softwares potentials of using abstraction to serve UI for visual-interactive systems outlined in *the Literature Review* Chapter led to justified requirement of **NF_01** – an evident, but necessary requirement of providing a user interface design. This was done in Max through the use of modular user interfaces and using object such dials, sliders and check-boxes. OpenFrameworks, an alternative implementation tool discussed in 2.6 of the *Literature Review* Chapter has the potential to include a user-interface, but it yields both limited and buggy functionality for this.

The requirement of **NF_02** stemmed from the issues raised in 2.4 and 2.5 of the *Literature Review* Chapter, where the importance of abstraction in computer-graphics and its importance within visual-interactive systems is discussed. The requirement was centred on the design of the system, and addressed by designing a modular system that used abstraction to hide complexity and to separate functionality whilst being able to execute operations within the real-time hybrid development environment of Max. Max provides a development environment identified to ensure high-end customisability, but it was necessary to research and understand the issues surrounding the program itself, including event priority outlined in 4.2.3 of the *Design* Chapter.

For real-time processing determined by **NF_03**, I chose Max as the appropriate tool fairly early on, that is capable of providing real-time processing for visual-interactive systems. Naturally, I also considered other languages, frameworks and software that could carry out this task too, such as OpenFrameworks and Cinder. These are explored more in 2.6 of the *Literature Review* Chapter. In order to be aware of real-time processing, I researched into the nature of Max's real-time processing outlined in 4.2 of the *Design* Chapter, which lead to an appreciation of the event priority that occurs in Max, which became useful for design considerations.

NF_04's requirement has the potential to provide more extended functionality for the end-user and increase the overall software potential for visual-interactive systems outlined in the research context in the *Introduction* Chapter at the beginning of this report. Other Max packages like Vizzie and BEAP can connect to these (Kluster's) modules, and other JavaScript and C++ externs can communicate with the modules. How this is accomplished is up to the end-user or developer's intentions.

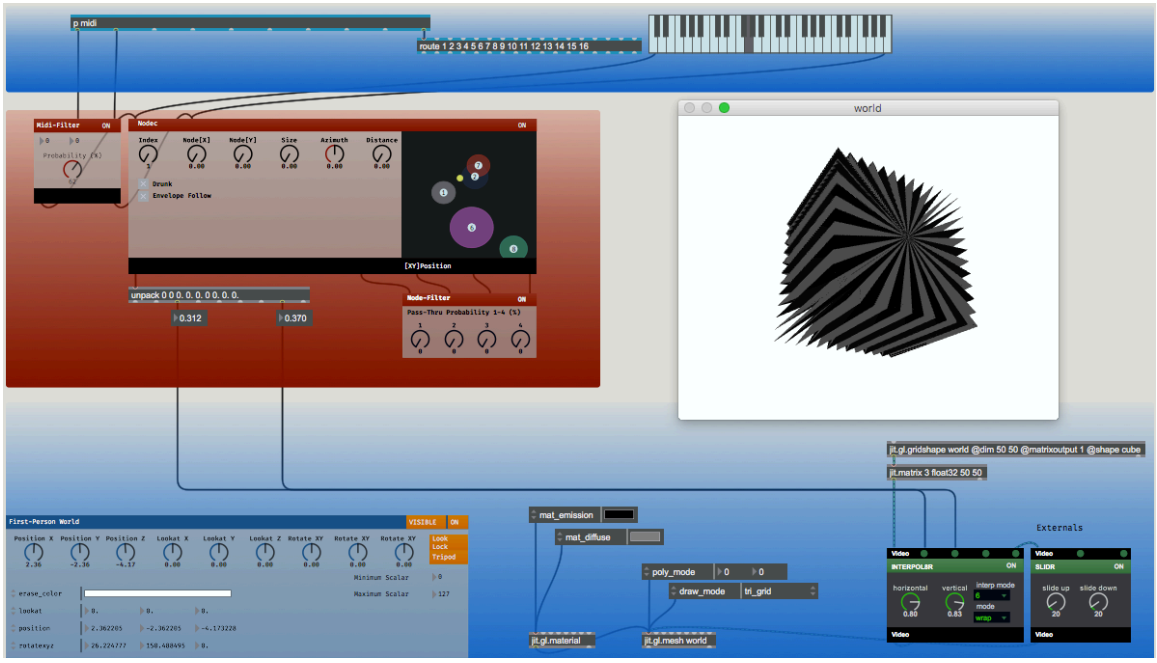


Figure 21: An illustration of the discussed modules in the *Implementation* Chapter. The coloured background panels represent the architectural boundaries of the system (blue for MIDI(top) red for data analysis, blue for graphics processing (bottom)). This example demonstrates interaction with externals for NF_04 (modules with green banner – Interpol8r and Slidr) and other internal Max objects ([jit.gl.mesh] and jit.gl.material] and the data object [unpack] which is a Max internal being used to unpack the Node list of evaluated functions). The window is visible using the ',' key, and rendering can be triggered on/off using the spacebar key.

In **NF_05,** evaluating whether a system uses power efficiently and is well defined at low costs was difficult to evaluate without replicating the exact same system and testing performance costs on the GPU and CPU. However, there are other ways in which performance was measured and low costs were ensured. Other software packages in Max such as Vizzie and BEAP, in which this project was inspired by, work efficiently within the Max, and I often observed the back-end of these modules to determine in what way the system could be designed and implemented to use power efficiently at low costs. Certain modules, notably the FpWorld module has a camera that can be driven externally or internally around a 3D world. On testing this module, I found that there was often a result of lag, especially when there was a high volume of matrices of dimensions of matrices being rendered in OpenGL.

6.3 DESIGN EVALUATION AND INTEGRATION

As mentioned in the design chapter, the system's conceptual and design was continuously redefined over the course of the project, and through the three main iterative phases of the project, a conceptual design eventually emerged naturally. Requirements of the complete system were not always clear, therefore working functionality and prototypes were often made to derive requirements through each phase to influence the design. This made it less costly than a SDLC like a Waterfall Model, as results from the system could be obtained early and periodically. Progress was always measured on a weekly basis during consultation with my supervisor, and testing and debugging were made easier.

In addition, the nature of changing requirements over the course of the project often left certain system architecture or design issues to arise, and at times the design of certain modules lost focus because certain phases resulted in an overall loss of definition of the complete system itself.

For these reasons, I acknowledged towards the end of the project that alternative design approaches could have been possible. Had the requirements been more detailed and solidified earlier on in the project, a V-model or an agile model might have resulted in a more manageable and better-documented system as a whole.

6.4 SUMMARY

The following chapter concludes this report by discussing possible future considerations in regard to the evaluated requirements this chapter set out. It concludes by mentioning the possible impact in relation to future considerations.

7 CONCLUSION

7.1 INTRODUCTION

This chapter covers the key future considerations that can be derived from the evaluation chapter, the impact, and a short reflection on difficulties that have arisen throughout the project.

7.2 IMPACT

The project's beneficiaries include the stakeholders outlined in the *Requirements*, some of whom may benefit from using the system to build new or extend interactive systems. The activities for successful impact involve sharing the research with those aiming to provide or produce visual-interactive systems. The research indicates that this could influence end-users approaches to decide to the design and implementation of future visual-interactive systems.

The project highlighted the effectiveness of using an iterative software design life cycle as a means to building interactive systems. However, I acknowledge that this claim cannot be made without replicating the system using a different software design life cycle such as a Waterfall or Agile model, and thoroughly comparing each model.

This project aims to highlight the importance of using high-level visual-programming languages such as Max rather than an existing code-based technology such as C++ to host a graphic library such as OpenGL. The benefits of adopting this approach are observed in the *Literature Review* Chapter. Again, it is important to acknowledge that this system should be replicated using an alternate approach to verify that this is the case.

Many of these impacts could be further validated and verified by the Embodied Audio-Visual Interactive Research Group (EAVI) at Goldsmiths, University of London outlined in the initial project brief, or any similar groups. It is important to note here that co-production may be a more effective way for this research to have impact, because the future considerations listed in 7.3 above indicate an increase in the projects scope.

7.3 FUTURE CONSIDERATIONS

Although there were many future considerations posed throughout the project, listed below are three broad, but realistic future considerations. These ultimately increase the scope of the project, and they aim to act as a foundation for future development. It is important to note that these considerations should be consulted in 7.2, when discussing the impact this project has outside academia.

The first consideration is replicating the system to further validate and verify existing requirements. This could involve replicating the system using a different design life cycle model and implementing the system using a different language such as C++. Comparing the two approaches proves valuable in demonstrating what the most suitable life-cycle model is for accelerating the construction of visual-interactive systems. A suitable alternative to this may project may be choosing a different software design life cycle model, and using OpenFrameworks or Cinder for implementation, discussed as an alternative implementation technology in the *Literature Review*.

The second consideration is producing a viable test plan for the system that covers tests on both a system and unit level. Whilst requirements-based testing was evaluated throughout each phase, there is more that can be done to verify the system's overall requirements. Performing automated unit tests in Max are tricky, and there is no recognised framework like JUnit framework for Java or GoogleTests

framework for C++. The only means of testing is by Max's debugging feature, by specifying suitable break-points to provide information to the console on what messages were being passed through the system and when. For higher level testing, a series of performance-based tests that are monitored through beta testing by the stakeholders would be an effective means of exposing bugs, and identifying new potential features of the system through a series of heuristic evaluations. At this stage of the project, it would also be useful for API and documentation to be written, so that specification-based testing can be performed to assess whether each module does what it says it does, and is feature-complete.

The third and final consideration is expanding existing requirements. This could be done by categorising requirements further as mentioned in the Requirements Practicalities in 3.6.1, and deriving a series of more focused requirements. FR_02 is a broad requirement, and more specific requirements would be more suitable for further analysis of the attributes of musical improvisation outlined in 3.3 of the *Requirements* Chapter. This could include requirements that perhaps design new or build upon existing modules to carry out more focused predictive analytics and machine and learning tasks that use not only MIDI, but data from graphics to influence OpenGL operations. The system could contain modules that have the potential to analyse both MIDI and OpenGL operations to provide a more complete analysis of the interactive system. This could be succeeded by an in-depth evaluation on successfully identified relationships between musical improvisation and OpenGL operations and any respective analysis tasks to explore notions of predictive analytics or supervised, reinforced or unsupervised machine learning.

7.4 REFLECTION

The first major difficulty that arose throughout this project involved continually defining appropriate limitations or constraints for the given time period. This was solved through weekly consultations with the project supervisor in order to maintain the focus on what was within the project's scope. The project is open-ended and the future considerations mentioned in 7.3 are broad, unearthing new research potential should any individual or myself wish to pursue this in the future.

The second difficulty was separating the scientific nature of this project from subjective constructs that often arouse debate in both music and computer-art. As mentioned in 2.5 of the Literature Review, there is a communication gap between designers and end-users, originated by their different cultural backgrounds.' [5] Nonetheless, a cautious Agile and/or Iterative approach from an experienced selection of the end-user types listed in 3.2 of the *Requirements* Chapter would be beneficial to the system, as long as the system was constantly re-evaluated using an assortment of evaluation techniques especially those that involve the use of visual-interactive installations and/or project work in performance scenarios.

8 BIBLIOGRAPHY

[1] Derek Bailey, *Improvisation: Its Nature and Practice in Music*. Chicago: DaCapo Press, 1993.

[2] Detlev Stalling, Malte Westerhoff, and Hans-Christian Hege, "Amira: a Highly Interactive System for Visual Data Analysis ," Zuse Institute Berlin (ZIB), Germany ,.

[3] Jonas Gomes and Luiz Velho, "Abstraction paradigms for computer graphics," *The Visual Computer*, vol. 11, no. 5, pp. 227-239, May 1995.

[4] Thomas Strothotte, *Computation Visualization: Graphics, Abstraction and Interactivity*.

[5] Maria Francesca Costabile, Daniela Fogli, Piero Mussio, and Antonio Piccinno, "Browse Journals & Magazines > IEEE Transactions on Systems. > Volume: 37 Issue: 6 Back to Results | Next > Visual Interactive Systems for End-User Development: A Model-Based Design Methodology," *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, vol. 37, no. 6, pp. 1029-1046, October 2007.

[6] openFrameworks. (2017, August) openFrameworks. [Online]. HYPERLINK "http://openframeworks.cc/documentation/" http://openframeworks.cc/documentation/

[8] Field Company Ltd., "Communion," La Gaite Lyrique in Paris, London,. https://www.field.io/project/communion/

[7] Memo Akten, "Laser Forest," http://www.memo.tv/laser-forest/.

[9] Cycling74. (2015, September) Cycling 74 Web site. [Online]. HYPERLINK "https://cycling74.com/tutorials/a-brief-introduction-to-vizzie" https://cycling74.com/tutorials/a-brief-introduction-to-vizzie

[10] Rebecca Fiebrink, "COMBINING D2K AND JGAP FOR EFFICIENT FEATURE WEIGHTING FOR CLASSIFICATION TASKS IN MUSIC INFORMATION RETRIEVAL ," Goldsmiths University, London, 2005.

[11] Baptiste Caramiaux and Atau Tanaka, "Machine Learning of Musical Gestures," in *Proceedings of the International Conference on New Interfaces for Musical Expression*, Oslo, 2013, pp. 513-518.

[12] Jamie Bullock, A machine learning library for Max and Pure Data, July 17, 2017, https://github.com/cmuartfab/ml-lib.

[13] Rebecca Fiebrink, P. R Cook, and D. Trueman, "Human model evaluation in interactive supervised learning." Proc. CHI," London, 2010.

[14] Matt Pearson, *Generative Art*. New York: Manning Publications Co., 2011.

[15] Hartmut Bohnacker, Benedikt Groß, Julia Laub, and Claudius Lazzeroni, *Generative Design: Visualize, Program and Create with Processing*.: Princeton University Press, 2012.

[16] Amineh Ghorbani, Pieter Bots, and Gerard Dijkema, "MAIA: a Framework for Developing Agent-Based Social Simulations," *Journal of Artificial Societies and Social Stimulation*, vol. 16, no. 2, 2013.

[18] Michael B. Spring. (2002) Encyclopedia. [Online]. HYPERLINK "http://www.encyclopedia.com/computing/news-wires-white-papers-and-books/interactive-systems" http://www.encyclopedia.com/computing/news-wires-white-papers-and-books/interactive-systems

[17] Mira Balaban, Kemal Ebcioglu, and Otto Laske, *Understanding Music with AI: Perspective on Music Cognition*, 1st ed., Mira Balaban, Kemal Ebcioglu, and Otto Laske, Eds. London; Cambridge: AAAI Press; MIT Press.

[19] P. Bottoni, Shi-Kuo Chang, M.F. Costabile, S. Levialdi, and P. Mussio, "Modeling visual interactive systems through dynamic visual languages," *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* , vol. 32, no. 6, pp. 654 - 669, November 2002.